Combined Multi-Agent Supervisor

2026-04-01

Method Goal

The combined notebook family is the most integrated RL-assisted method in the repo. It runs three decision layers around one MPC solve:

- a DQN agent chooses the horizon recipe,
- a TD3 agent scales the internal model matrices,
- a second TD3 agent scales the MPC cost weights.

The plant still receives a standard MPC move, but that move is generated by an MPC object whose horizons, model, and weights may all change online.

Notebook Sources Covered

- `RL_assisted_MPC_combined.ipynb`
- `RL_assisted_MPC_combined_dist.ipynb`
- Shared helpers from `utils/helpers.py`, `DQN/dqn_agent.py`, and `TD3Agent/agent.py`

State, Actions, Reward

All three agents see the same scaled state:

$$s_t = \begin{bmatrix} \tilde{x}_t \\ \tilde{y}_{\mathrm{sp},t} \\ \tilde{u}_t^{\mathrm{dev}} \end{bmatrix}.$$

The actions are heterogeneous:

- DQN horizon action: $a_t^H \mapsto (H_p^{(t)}, H_c^{(t)})$
- TD3 model action: $a_t^M \mapsto [\alpha_t, \delta_t]$

- TD3 weight action: $a_t^W \mapsto [m_{Q_1}, m_{Q_2}, m_{R_1}, m_{R_2}]$

All three agents are trained from the same scalar reward after the MPC move has been applied and the plant output has been observed.

Core Algorithm Flow

1. Build the common scaled RL state.
2. Select or reuse a horizon action; only refresh it every `decision_interval`.
3. Select model multipliers and build fresh (A, B) from immutable baselines.
4. Select weight multipliers and update (Q, R).
5. Rebuild the MPC object if the horizon pair changed, then overwrite its active (A, B, Q, R) values regardless.
6. Solve MPC, apply the first move, step the plant, and update the observer.
7. Push transitions into whichever agents are active and call each `train_step()` after warm start.

Key Code Paths

Notebook-local combined decision logic

```
if agent_horizon is None:
    h_idx = horizon_baseline_idx
    Hp, Hc = current_Hp, current_Hc
else:
    if i <= WARM_START:
        h_idx = horizon_baseline_idx
    else:
        if (i % decision_interval == 0) or (last_h_idx is None):
            ...
        Hp, Hc = action_to_horizons(horizon_recipes, h_idx)

m_model = map_to_bounds(a_model_raw, model_low, model_high)
alpha = float(m_model[0])
delta = np.asarray(m_model[1:1 + n_inputs], float).reshape(-1)

m_w = map_to_bounds(a_w_raw, weights_low, weights_high)
Q_now = Q_base.copy()
R_now = R_base.copy()
Q_now[0] *= float(m_w[0])
Q_now[1] *= float(m_w[1])
R_now[0] *= float(m_w[2])
R_now[1] *= float(m_w[3])
```

Notebook-local MPC rebuild and logging

```
if (int(Hp), int(Hc)) != (current_Hp, current_Hc):
    rebuild_mpc(Hp, Hc, A_now.copy(), B_now.copy(),
                Q_now.copy(), R_now.copy())
    current_Hp, current_Hc = int(Hp), int(Hc)

MPC_obj.A = A_now
MPC_obj.B = B_now
MPC_obj.Q_out = Q_now
MPC_obj.R_in = R_now

horizon_trace[i, :] = (current_Hp, current_Hc)
alpha_log[i, 0] = alpha
delta_log[i, :] = delta
weight_log[i, :] = m_w
```

Notebook-local multi-agent training block

```
if not test:
    if agent_model is not None and i > WARM_START:
        agent_model.push(s, a_model_raw.astype(np.float32), r, s2, float(done))
        if i >= warm_start:
            agent_model.train_step()

    if agent_weights is not None and i > WARM_START:
        agent_weights.push(s, a_w_raw.astype(np.float32), r, s2, float(done))
        if i >= warm_start:
            agent_weights.train_step()

    if agent_horizon is not None and i > WARM_START:
        agent_horizon.push(s, int(h_idx), r, s2, float(done))
        if i >= warm_start:
            agent_horizon.train_step()
```

Variants Covered

- The nominal notebook runs the combined supervisor without gradual process drift.
- The disturbance notebook injects the shared Q_i, Q_s, and hA schedules while reusing the same combined architecture.
- Both notebooks log horizon selections and continuous multiplier traces so the three agent families can be analyzed together.

Limitations, Caveats, And Open Questions

- The observer gain refresh hook exists in the function signature through `observer_update_interval`, but the actual observer-gain update block is currently commented out in the combined note-

book.

- Baseline fallback behavior before warm start is part of the method and should not be removed casually.
- All three learners share the same scalar reward, so credit assignment is implicit rather than decomposed.
- This method depends on notebook-local orchestration and should be reviewed there first, not in legacy helper modules.